

4. Продвинутые приемы разработки сценариев

То, что мы рассмотрели в предыдущих главах, касается базовых команд сценариев, выполняющих достаточно элементарные операции. Основываясь на этих знаниях, можно создавать достаточно сложные сценарии. Однако в полной мере возможности сценарного движка раскрываются при создании собственных функций.

Для этого необходимо более детальное погружение в алгоритмы работы сценариев в RRS.

4.1. Порядок выполнения скрипта сценария

Как выполняется скрипт сценария? Тот файл, что мы создавали на протяжении трех глав, `main.lua`, загружается при запуске игры и выполняется **полностью** от начала и до конца. Все записанные туда команды так или иначе воздействуют на движок игры, заставляя её выполнять определенные действия. Одни команды выполняются сразу же, другие - регистрируются симулятором и ждут своего часа. Разберем это подробнее.

1. Команды, выполняемые при загрузке игры

Это все, что касается начальной инициализации поездки

- `setTime`, `setDate`, `setDateTime` - устанавливают дату при запуске игры
- `setTrain` - устанавливает заданный поезд в заданное место карты. Это тоже выполняется при запуске игры

Все остальные команды имеют отложенное действие.

2. Команды, выполняемые в процессе игры

Команды установки триггеров добавляют триггер в общий список триггеров, и в процессе исполнения игры происходит периодическая проверка всех триггеров на соответствие заданных триггером условий. При совпадении этих условий у какого-то из триггеров (например наступление заданного момента времени) вызывает срабатывание `action`-функции, которую вы указали в качестве действия для этого триггера.

Временные триггеры, то есть те, которые срабатывают на заданное время, проверяются периодически на каждом шаге симуляции. Сработавший триггер удаляется из списка.

Позиционные триггеры проверяются по наступлении события "изменение состояния траектории". То есть когда где-либо на карте происходит либо занятие, либо освобождение траектории. Данные триггеры могут удаляться после срабатывания, а могут оставаться в системе и срабатывать снова и снова. Существует доступный разработчику сценария механизм управления временем жизни триггера.

4.2. Создание собственной `action`-функции

В [приложении 2](#) приведен список готовых `action`-функций, которые можно указывать в качестве действий для триггеров. Однако, этого функционала может не хватить, поэтому полезно научиться создавать свои `action`-функции.

Итак, `action`-функция - это функция на языке Lua. И выглядит она так

```
function my_action()

    -- Здесь пишем свою произвольную логику

end
```

Здесь:

- function - ключевое слово Lua, означающее начало описания функции
- my_action - имя функции, оно произвольное, но удовлетворяет ряду требований: латиница, не должно начинаться с цифры, не должно содержать пробелов, арифметических знаков, слешей, кавычек, скобок.
- () - в круглых скобках - список параметров, передаваемых в функцию. В данном примере нет параметров.
- end - ключевое слово, обозначающее конец тела функции.

Внутри функции можно описать произвольные действия, например вызвать системную функцию ([приложение 1](#)) построения поездного маршрута

```
function my_action()

    buildTrainRoute("track_a_p2b", "track_a-b_nd-22", 1)

end
```

Теперь нашу action-функцию мы можем передать в триггер, например во временной

```
setTimeTrigger("17:30:10", my_action)
```

Полный текст сценария, содержащего эти действия, таков

```
-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p2b"
my_train.coord = 1690.0
my_train.dir = 1

-- Устанавливаем поезд игрока
setTrain(my_train)

-- Пользовательская action-функция
function my_action()

    buildTrainRoute("track_a_p2b", "track_a-b_nd-22", 1)

end
```

```
-- Ставим временной триггер с пользовательским action
setTimeTrigger("17:30:10", my_action)
```

То же самое действие, но с использованием стандартной action-функции `actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1)` мы делали в [главе 2](#), что гораздо быстрее и проще. В чем разница?

Разница в том, что в нашей функции мы можем определить произвольные действия, например построить не один, а два маршрута

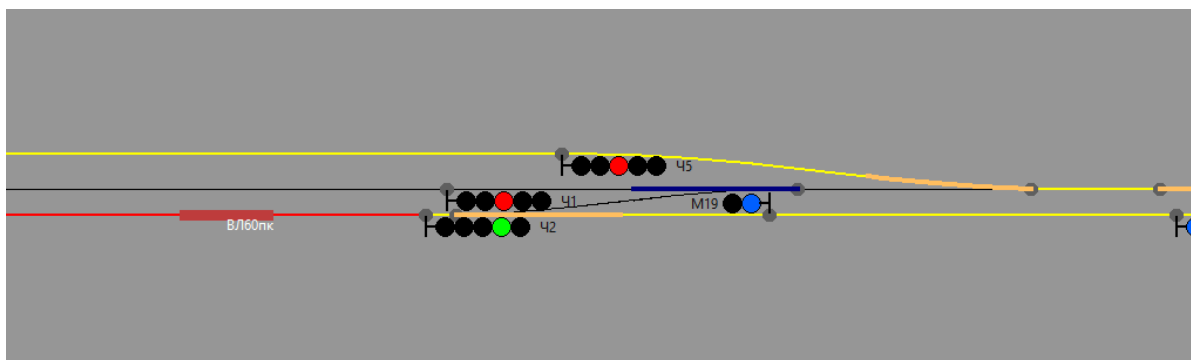
```
function my_action()

    -- Строим четный маршрут отправления
    buildTrainRoute("track_a_p2b", "track_a-b_nd-22", 1)

    -- Строим нечетный маршрут приема
    buildTrainRoute("track_a-b_n-1", "track_a_p5b", -1)

end
```

Видоизменив функцию, мы получаем следующую картину



Логика работы нашей action-функций может быть произвольной. К сожалению, для полного овладения всеми возможностями, которые предоставляет данный механизм, придется погрузиться в изучение языка Lua. Но некоторые вещи мы все же разберем здесь.

4.3. Передача параметров в action-функцию

В предыдущем примере мы сконструировали action-функции без параметров, а в вызываемые внутри системные функции передали конкретные имена траекторий и направление.

Однако, когда создается функция, обычно это подразумевает некоторые универсальные операции, выполняемые многократно.

Рассмотрим такую задачу - поставить временной триггер на построение сквозного пропуска в двух направлениях для произвольной станции. Причем action-функция будет одна единственная, а триггеров мы поставим два - один для станции В, другой для станции С.

Для сквозного пропуска нам надо знать имена участков приближения и удаления для каждой станции в каждом направлении. Это мы, разумеется, выясним, но прежде напишем универсальную action-функцию, строящую маршруты в двух направлениях.

```
-- Построение двух сквозных маршрутов пропуска
function dual_approach(traj_CHP1, traj_CHU1, traj_NP1, traj_NU1)
```

```

-- Объявляем функцию БЕЗ параметров
function action()

    -- Строим маршрут от траектории traj_CHP1 до traj_CNU1 в направлении 1
    buildTrainRoute(traj_CHP1, traj_CNU1, 1)
    -- Строим маршрут от траектории traj_NP1 до traj_NU1 в направлении -1
    buildTrainRoute(traj_NP1, traj_NU1, -1)

end

-- Возвращаем функцию
return action

end

```

Функция `dual_approach` будет строить два встречных сквозных маршрута по станции, и она имеет четыре параметра. Параметры функции указываются в скобках, через запятую. Имена параметров подчиняются тем же правилам, что и имена функций. Смысл данных параметров следующий

- `traj_CHP1` - первый четный участок приближения (имя траектории)
- `traj_CNU1` - первый четный участок удаления
- `traj_NP1` - первый нечетный участок приближения
- `traj_NU1` - первый нечетный участок удаления

Однако, видно, что внутри функции `dual_approach` у нас описана еще одна функция, без параметров. Дело в том, что движок игры, для универсальности организации таких функций, полагает их функциями без параметров. Поэтому нам нужна внешняя функция, с параметрами - именами участков приближения и удаления, и внутренняя функция, без параметров, которая вызовет системные функции построения маршрута, передав им параметры внешней функции.

Обратите внимание на эту строку

```

-- Возвращаем функцию
return action

```

Ключевое слово `return` означает, что функция возвращает некоторое значение, результат своей работы, который может быть использован в дальнейшем. В данном случае мы возвращаем *функцию*. Внутри функции `dual_approach` мы конструируем `action`-функцию, используем для её логики переданные параметры, а потом возвращаем эту функцию как результат работы.

А далее мы пишем вот так

```

setTimeTrigger("17:30:20", dual_approach("track_a-b_2-ch", "track_b-c_nd-22",
"track_b-c_n-1", "track_a-b_21-chd"))

```

Ставим триггер на время 17:30:20, а в качестве `action`-функции передаем результат работы функции `dual_approach`, которая создает `action`-функцию для триггера с нужными нам параметрами.

В данном случае в 17:30:20 будут построены два сквозных маршрута по станции В. Если мы хотим проделать то же самое и для станции С, поставим еще один триггер

```
setTimeTrigger("17:30:20", dual_approach("track_b-c_2-ch", "track_c_m3-nd",  
"track_c_m1-n", "track_b-c_21-chd"))
```

Здесь мы изменили только имена участков приближения и удаления, остальная логика осталась неизменной.

Такой механизм, когда мы конструируем функцию "на ходу", передаем её для использования другой функции, и при этом она может выполняться не прямо сейчас, а гораздо позже, по наступлению определенного события, носит название "анонимная функция", или лямбда-функция. Такой подход широко используется в скриптовом языке Lua, движок сценариев RRS активно его эксплуатирует.

4.4. Триггеры, устанавливающие триггеры, устанавливающие триггеры...

С помощью action-функций можно добиться того, что, например, установленный нами триггер на одно событие может сам установить триггер уже на другое событие. Это позволяет создать в сценарии цепочку взаимосвязанных друг с другом действий, не обязательно четко привязанных к конкретному времени.

Например, рассмотрим такую ситуацию. На перегоне, по направлению к станции В, один за другим следуют два поезда. Первый поезд мы принимаем на боковой путь, а второму - организуем сквозной пропуск. Затем, когда второй поезд проследует по станции В, нам надо отправить первый поезд по удалению за вторым. То есть мы ставим первый поезд под обгон вторым, а все операции по построению маршрутов хотим возложить на сценарий.

Первым вариантом решения такой задачи могло бы быть создание временных триггеров, которые отработают последовательно на построение маршрутов. Но кто дает гарантию, что мы угадаем моменты времени, когда следует строить тот или другой маршрут? Разумеется, поезда должны следовать по графику, и определенные временные рамки всех перечисленных операций у нас имеются. Но мы не можем и не должны полагаться на это! В реальной жизни, дежурный по станции прежде всего готовит маршруты поезда, исходя из свободности и занятости участков станции, а соблюдение графика складывается в целом из его грамотных действий и грамотной работы машинистов поездов.

Поэтому мы воспользуемся позиционными триггерами. Пока попробуем обойтись стандартными триггерами ([приложение 3](#)) и собственными action-функциями, там, где это действительно требуется.

Итак, создадим два пассажирских поезда, для ускорения демонстрации примера расположим их на перегоне, на достаточном удалении друг от друга, так, чтобы их разделяло два блок-участка. Первый поезд поставим перед вторым участком приближения к станции В с четного направления.

```
-- Устанавливаем поезд 1  
train1 = TrainData.new()  
train1.name = "1"  
train1.config = "v160pk-1543-T65_17"  
train1.traj = "track_a-b_6-4"  
train1.coord = 1900.0  
train1.dir = 1  
  
setTrain(train1)
```

```
-- Устанавливаем поезд 3
train3 = TrainData.new()
train3.name = "3"
train3.config = "v160pk-1543-T65_17"
train3.traj = "track_a-b_12-10"
train3.coord = 1900.0
train3.dir = 1

setTrain(train3)
```

Первый поезд мы принимаем на 4 путь. Маршрут ему надо построить, когда он вступит на второй участок приближения к станции В с четного направления. Имя траектории, которая является этим участком, мы можем определить из карты, но для удобства запишем его в переменную

```
-- Второй участок приближения к станции В
local stB_CHP2 = "track_a-b_4-2"
```

Это имя мы будем использовать несколько раз, поэтому его удобно записать в локальную переменную с именем stB_CHP2. Ключевое слово local означает, что данная переменная видна только внутри скрипта main.lua.

Заведем переменные для других траекторий, занятие и освобождение которых мы будем отслеживать. Они же будут участвовать в построении маршрутов. То есть опять-таки удобно объявить эти имена как переменные

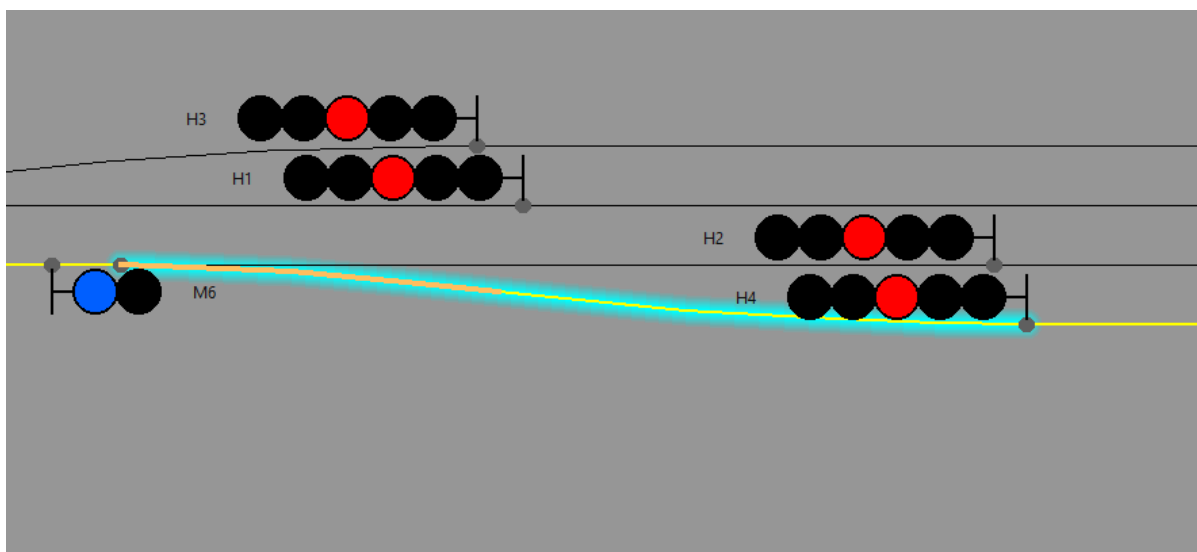
```
-- Первый участок удаления от станции В
local stB_CHU1 = "track_b-c_nd-22"
-- Второй участок удаления от станции В
local stB_CHU2 = "track_b-c_22-20"
-- 4 путь станции В
local stB_p4 = "track_b_p4"
-- Стрелочный участок от стрелки 12 до 4 пути станции В
local sw12_n4 = "track_b_12-p4"
```

Построим маршрут приема для поезда 1, по факту его вступления на второй участок приближения

```
-- Строим маршрут приема поезду 1 на 4 путь станции В
setOnTrajBusyTrigger(stB_CHP2, actionBuildTrainRoute(stB_CHP2, stB_p4,
train1.dir))
```

Используем стандартный триггер, срабатывающий по занятию указанной траектории, и стандартный action построения поездного маршрута, от второго участка приближения, до 4 пути. Вместо конкретных названий траекторий используем *переменные*, в которых хранятся данные имена.

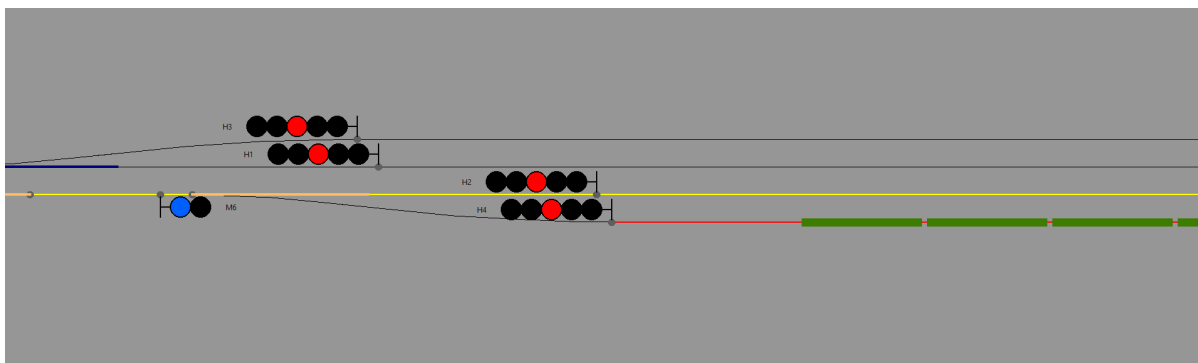
Когда нам следует построить маршрут для поезда 3? Тогда, когда это станет возможным, а именно когда поезд 1 не будет уже занимать необходимые для построения этого маршрута стрелки. Например, когда поезд 1 полностью заедет на 4 путь, а значит он освободит стрелочный участок, примыкающий к этому пути, то есть траекторию, показанную на рисунке



Имя этой траектории мы записали в переменную sw12_H4. По факту её освобождения мы можем построить маршрут пропуска поезду 3, например так

```
setOnTrajFreeTrigger(sw12_H4, actionBuildTrainRoute(stB_CHP2, stB_CHU1,
train3.dir))
```

При текущем расположении поездов это безусловно сработает - как только поезд 1 освободит стрелочный участок, заехав на 4 путь, маршрут поезду 3 будет построен.



Но что, если дистанция между поездами будет другой? Если поезда будут ближе друг к другу, вплоть до ситуации, когда они занимают два подряд блок участка, работа сигнализации на перегоне не даст им сблизиться на дистанцию менее длины одного блок участка. Но все равно, чтобы избежать ситуации блокировки построения маршрута приема/пропуска, лучше строить его не от второго, а от первого участка приближения

```
-- Первый участок приближения к станции В
local stB_CHP1 = "track_a-b_2-ch"
```

и для маршрутов перепишем триггеры

```
-- Строим маршрут приема поезду 1 на 4 путь станции В
setOnTrajBusyTrigger(stB_CHP2, actionBuildTrainRoute(stB_CHP1, stB_p4,
train1.dir))

-- Строим маршрут пропуска поезду 3
setOnTrajFreeTrigger(sw12_H4, actionBuildTrainRoute(stB_CHP1, stB_CHU1,
train3.dir))
```

Если расстояние между поездами будет более значительным, может пройти достаточно большое время, прежде чем потребуется маршрут пропуска для поезда 3. И если мы построим маршрут сразу, то главный путь и все сопряженные с ним участки станции будут заняты до тех пор, пока поезд 3 не будет пропущен по маршруту. Поэтому лучше, в данной ситуации, строить маршрут поезду 3 так же, как и для поезда 1 - по факту занятия им второго участка приближения. А для этого нам нужно установить триггер на построение этого маршрута уже после того, как поезд 1 освободит стрелочный участок. Напишем для этого собственную action-функцию

```
-- Действия, выполняемые при освобождении поездом 1 стрелочного участка 12 - путь 4
function set_appr_train3()

    -- Строим маршрут пропуска поезду 3
    setOnTrajBusyTrigger(stB_CHP2, actionBuildTrainRoute(stB_CHP1, stB_CHU1,
train3.dir))

end
```

Эта функция выполняет операцию установки нового позиционного триггера на занятие второго участка приближения, но уже с построением сквозного маршрута для поезда 3. Эту функцию мы подставим в качестве action для триггера на освобождение стрелочного участка перед 4 путем

```
setOnTrajFreeTrigger(sw12_n4, set_appr_train3)
```

Теперь при освобождении стрелочного участка маршрута не будет строиться маршрут пропуска, а будет установлен триггер, который сработает при занятии поездом 3 второго участка приближения и построит ему маршрут.

И наконец, чтобы завершить обгон, нам надо построить маршрут поезду 1. Мы сможем это сделать, когда поезд 3 покинет хотя бы первый, а лучше - второй участок удаления. Делаем эту операцию путем установки позиционного триггера на освобождение второго участка удаления

```
-- Строим маршрут отправления поезду 1 с 4 пути, когда освободится второй участок
удаления
setOnTrajFreeTrigger(stB_CHU2, actionBuildTrainRoute(stB_p4, stB_CHU1,
train1.dir))
```

И на этом примере продемонстрируем еще один временной триггер, устанавливаемый функцией `setPostEventTimeTrigger()` (см. [приложение 1](#))

Этот триггер срабатывает по времени, *через промежуток времени, прошедший с момента его установки.*

Например, мы хотим построить маршрут отправления поезду 1 не сразу после того, как поезд 3 освободит второй участок удаления, а через 1 минуту 20 секунд после этого. Для этого создадим свою action-функцию


```
-- Действия, выполняемые после освобождения поездом 3 второго участка удаления
function set_dep_train1()

    -- Строим маршрут отправления поезду 1 через 1 минуту 20 секунд после
    освобождения ЧУ2
    setPostEventTimeTrigger("+00:01:20", actionBuildTrainRoute(stB_p4, stB_CHU1,
    train1.dir))

end
```

и устанавливаем её в качестве действия на триггер освобождения второго участка удаления

```
setOnTrajFreeTrigger(stB_CHU2, set_dep_train1)
```

Таким образом, при использовании собственных action-функций можно комбинировать различные триггеры, устанавливая их в том числе и каскадно, создавая достаточно сложные цепочки событий и действий по управлению движением из сценария.

4.5. Внутренняя организация позиционного триггера.

Функция триггера. Время жизни позиционного триггера.

Те позиционные триггеры, о которых мы до сих пор говорили, описанные в [приложении 3](#), являются стандартными и обрабатывают типовые ситуации. Эти триггеры работают поверх системного слоя симулятора, реализованы на языке Lua, а их реализацию можно увидеть в файле [modules/lua/triggers fabric.lua](#). В него не возбраняется заглянуть и увидеть, что реализация достаточно сложная. Возьмем для примера функцию setOnTrajFreeTrigger()

```
-----
-- Занятие траектории произвольным поездом
-----
function setOnTrajBusyTrigger(name_traj, action)

    function on_busy_trigger_func(train_name, traj_name, is_busy)

        if is_busy and traj_name == name_traj then

            action()

            return TRIG_DELETE
        end

        return TRIG_SAFE
    end

    setTrigger(on_busy_trigger_func)
end
```

Напомню, что setOnTrajFreeTrigger - это функция, которая устанавливает триггер (событие), срабатывающий при занятии заданной траектории произвольным поездом. В эту функцию передается два параметра: имя траектории (name_traj), занятие которой проверяется, и функция, которая должна при этом выполняться (action).

Внутри реализации описана еще одна функция

```
function on_busy_trigger_func(train_name, traj_name, is_busy)

    if is_busy and traj_name == name_traj then

        action()

        return TRIG_DELETE
    end

    return TRIG_SAFE
end
```

Это - так называемая *функция позиционного триггера*. Она может иметь любое разрешенное имя, но имеет четкий список параметров, порядок которых нельзя нарушать

- train_name - имя поезда, занявшего или освободившего траекторию
- traj_name - имя траектории, которая была занята или освобождена
- is_busy - состояние занятости траектории: true - занята, false - свободна

Каждый раз, когда в процессе игры происходит изменение состояния какой-либо траектории, движок симулятора обращается к списку позиционных триггеров, за каждым из которых закреплена такая вот функция. Симулятор вызывает эти функции по очереди и в *каждую из них* передает имя поезда, изменившего состояние траектории, имя траектории, которая поменяла свое состояние, и то состояние (is_busy), которая она получила.

То есть сам симулятор вызывает эту функцию и передает туда перечисленную информацию. И дальше дело за разработчиком, как использовать её.

В данном примере стандартного триггера происходит проверка **одновременного выполнения** двух условий

- траектория была занята (is_busy - истинно)
- имя занятой траектории совпадает с той, что мы передали в функцию установки триггера (traj_name == name_traj)

В случае, если оба условия выполнены, значит наступило то событие, которое мы ожидаем, и тогда выполняется функция action(), которая, как мы видели выше, может содержать произвольную последовательность действий. После этого следует выход из функции, с передачей симулятору кода выхода TRIG_DELETE

```
return TRIG_DELETE
```

Симулятор, приняв код выхода TRIG_DELETE, понимает, что данный триггер больше не нужен, и удаляет его из списка обрабатываемых триггеров.

Если хотя бы одно из этих условий неверно, функция action() не выполняется. И мы выходим из функции, передав симулятору другой код выхода - TRIG_SAFE, что указывает на то, что триггер следует сохранить в списке. В этом случае он будет проверяться на срабатывание снова и снова, при каждом изменении состояния траекторий в игре.

Операция возврата кода выхода - обязательна при реализации функции триггера. Симулятор ожидает получить указание о том, что делать с триггером, после того как будет вызвана и отработает данная функция. Таким способом, логика внутри триггерной функции может управлять временем жизни позиционного триггера.

В конце функции `setOnTrajBusyTrigger` содержится вызов

```
setTrigger(on_busy_trigger_func)
```

`setTrigger` - это системная функция (см. [приложение 1](#)) установки позиционного триггера. В качестве единственного параметра она принимает функцию триггера, в данном случае это функция `on_busy_trigger_func`, определенная выше. Это явное указание симулятору добавить данный триггер в список контролируемых позиционных триггеров.

ВАЖНО!!! Функция позиционного триггера имеет строго определенный формат!

```
function pos_trig_func(train_name, traj_name, is_busy)

    -- Здесь пишем произвольную логику, которая может использовать
    -- параметры train_name, traj_name, is_busy, получаемые от симулятора

    return TRIG_SAFE

end
```

Эта функция обязательно имеет три параметра: имя поезда, имя траектории, состояние, которое приняла траектория. Именно в этом порядке!

Эта функция обязательно должна возвращать симулятору код выхода!

Для установки позиционного триггера, обрабатываемого через данную функцию, обязательно выполняем вызов

```
setTrigger(pos_trig_func)
```

Это - еще один инструмент расширения функциональности движка сценариев. Да, предусмотрены стандартные триггеры на занятие/освобождение траекторий (см. [приложение 3](#)). Возможно, по результатам опробования системы сценариев пользователями, данный список будет расширен. Однако, все возможные ситуации предусмотреть нереально.

Ниже я приведу пример того, как это можно использовать на практике. Но предварительно мы рассмотрим один небольшой вопрос.

4.6. Некоторые сервисные функции, предоставляемые игрой

Допустим, наша логика предусматривает необходимость проверки состояния какого-либо участка станции или перегона, и от этого будет зависеть то, как будет работать сценарий дальше. Для получения состояния траектории в любой момент предусмотрена системная функция `getTrajState` (см. [приложение 1](#)). Эта функция принимает на вход имя траектории и возвращает состояние этой траектории в данный момент.

Пример использования

```
local traj_state = getTrajState("track_a_p1a")
```

Данная конструкция проверит состояние траектории с именем "track_a_p1a" и запишет его в переменную traj_state. Переменная traj_state является *структурой* и содержит два параметра

- is_busy - признак того, что траектория занята (true - занято, false - свободно)
- in_route - признак того, что данная траектория включена в уже построенный маршрут

Дальнейшее использование результата работы этой функции предусматривает применение условий, например

```
-- Проверяем занятость траектории
if traj_state.is_busy then

    -- Действия, которые мы предпримем, если траектория занята

end

-- Проверяем включение траектории в маршрут
if traj_state.in_route then

    -- Действия, которые мы предпримем, если траектория включена в маршрут

end
```

Еще одна функция, которая может быть полезна - getNextTraj. Она принимает два параметра - имя траектории и направление (1 или -1), и возвращает имя траектории, которая следует за данной траекторией в указанном направлении. Если траектория в заданном направлении оканчивается стрелкой, то вернется имя той траектории, на которую переключена стрелка в данный момент.

Пример использования:

```
local next_traj = getNextTraj("track_a_p1a", 1)
```

в переменную next_traj будет записано имя траектории, которая следует за траекторией "track_a_p1a" в направлении 1 ("туда").

Следующая функция может быть полезна при отладке сценариев. Эта функция выводит произвольное текстовое сообщения в лог симулятора logs/journal.log

```
logMessage("hello from RRS scenario system!")
```

Это может помочь разработчику понять, какие части его сценария выполняются и в какое время. Данные сообщения содержат строку "LUA DEBUG:", которая поможет вам отделить их от достаточно большого количества информации, выводимой в лог при работе игры.

4.7. Практическое применение функции позиционного триггера

Представим себе ситуацию, когда мы ставим триггер на занятие второго участка приближения, рассчитывая построить маршрут пропуска для поезда. Хорошо, если все необходимые участки станции будут при этом свободны. Но может ли оказаться иначе? Разумеется. Например, горловина станции может быть занята другим зазевавшимся поездом. Или участки, через которые будет пролегать маршрут пропуска, будут включены в другой маршрут, например маршрут отправления другого поезда.

Разумеется, если движение в сценарии организуется по графику, такие ситуации могут и не возникнуть, но гарантировать этого нельзя. Покажем, как средствами сценария можно подобную историю грамотно обработать.

Подготовим новый сценарий для всего этого безобразия

```
-- Устанавливаем поезд 102
train102 = TrainData.new()
train102.name = "102"
train102.config = "vl60pk-1543-T65_17"
train102.traj = "track_b_p4"
train102.coord = 1250.0
train102.dir = 1
train102.auto = true

setTrain(train102)

-- Устанавливаем поезд 104
train104 = TrainData.new()
train104.name = "104"
train104.config = "vl60pk-1543-T65_17"
train104.traj = "track_a-b_6-4"
train104.coord = 1900.0
train104.dir = 1
train104.auto = true

setTrain(train104)

-- Второй участок приближения к станции В
local stB_CHP2 = "track_a-b_4-2"
-- Первый участок приближения к станции В
local stB_CHP1 = "track_a-b_2-ch"
-- Первый участок удаления от станции В
local stB_CHU1 = "track_b-c_nd-22"
```

Поезд 104 находится рядом со вторым участком приближения. Но мы бездумно построим маршрут отправления поезду 102, который стоит на 4 пути станции

```
setTimeTrigger("+00:00:36", actionBuildTrainRoute(train102.traj, stB_CHU1,
train102.dir))
```

Пока поезд 104 доедет до второго приближения, маршрут уже будет построен. Соответственно горловина станции В будет занята, и сквозной маршрут этому поезду построить не выйдет. Однако, создав проблему, мы героически решим её с помощью такой функции для позиционного триггера

```
-- Специальная функция для позиционного триггера
function train104_appr(train_name, traj_name, is_busy)

    -- Если занята траектория stB_CHP2
    if is_busy and traj_name == stB_CHP2 then

        -- Проверяем состояние участка удаления
        local traj_state = getTrajState(stB_CHU1)

        -- Если он занят или включен в маршрут
        if traj_state.is_busy or traj_state.in_route then

            -- Ставим триггер на его освобождение с целью таки построить маршрут
            setOnTrajFreeTrigger(stB_CHU1, actionBuildTrainRoute(stB_CHP1,
stB_CHU1, train104.dir))

        else

            -- Если всё ок, строим маршрут
            buildTrainRoute(stB_CHP1, stB_CHU1, train104.dir)

        end

        -- В любом случае триггер удаляем, он отработал
        return TRIG_DELETE

    end

    -- Оставляем триггер, если он не сработал
    return TRIG_SAFE

end
```

Триггер срабатывает на условие занятия второго участка приближения

```
-- Если занята траектория stB_CHP2
if is_busy and traj_name == stB_CHP2 then
```

Однако, не спешим строить маршрут, для начала проверим состояние первого участка удаления

```
-- Проверяем состояние участка удаления
local traj_state = getTrajState(stB_CHU1)
```

Далее, если он или занят, или участвует в другом маршруте

```
if traj_state.is_busy or traj_state.in_route then
```

мы ставим триггер на его освобождение, и уже этот триггер построит для поезда 104 маршрут пропуска, когда поезд 102 уберется с первого удаления

```
-- Ставим триггер на его совобожение с целью таки построить маршрут
setOnTrajFreeTrigger(stB_CHU1, actionBuildTrainRoute(stB_CHP1,
stB_CHU1, train104.dir))
```

Если же вдруг участок удаления окажется свободен, просто построим маршрут пропуска

```
-- Если всё ок, строим маршрут
buildTrainRoute(stB_CHP1, stB_CHU1, train104.dir)
```

Отработавший триггер, независимо от занятости траектории удаления, мы приказываем убрать из системы

```
-- Влюбом случае триггер удаляем, он отработал
return TRIG_DELETE
```

Оставляем триггер только в том случае, если второй участок приближения еще не занимался

```
-- Оставляем триггер если он не сработал
return TRIG_SAFE
```

Не забываем установить нашу триггерную функцию в системе

```
setTrigger(train104_appr)
```

Надо сказать, что данный пример довольно синтетический, однако он иллюстрирует, что функция позиционного триггера может быть написана самостоятельно и содержать довольно развесистую логику.